

Université Paris Nanterre

Module : Open Data

Rapport de Projet Open Data

Modélisation et simulation de la propagation d'un virus

Présenté par :

**Melissa AKLI
Hanna DOUER**

Code source GitHub :

<https://github.com/Melissa-akli25/OpenData.git>

Encadré par :

Valentin Bouquet

Année Universitaire 2025-2026

Table des matières

Introduction	3
1 Description du projet et objectifs	5
1.1 Objectif principal	5
1.2 Fonctionnement global	5
2 Bibliothèques, Outils et Technologies	8
3 Travail réalisé	9
3.1 Création de l'environnement de simulation	9
3.2 Génération automatique de la base de données	9
3.3 Gestion des agents avec Mesa	10
3.4 Déplacements intelligents et modélisation avec NetworkX	10
3.5 Implémentation du modèle SEIR	11
3.6 Détection des interactions et contaminations	11
3.7 Visualisation graphique avec Pygame	11
3.8 Export des données au format CSV	11
4 Détails Techniques (Analyse des fichiers)	12
4.1 Fichier graph.js	12
4.2 Fichier map.js	12
4.3 Fichier pathfinding.js	12
4.4 Fichier renderer.js	13
4.5 Fichier simulation.js	13
4.6 Fichier ui.js	13
4.7 Fichier main.js	13
4.8 Scripts Python (Génération et Conversion)	13
4.9 Fichier index.html	13
5 Bilan	14
5.1 Difficultés rencontrées	14
5.2 Perspectives et améliorations possibles	15
5.3 Simulations supplémentaires possibles	15
Bibliographie	16
Annexes	17

Conclusion	18
5.4 Conclusion	18

Introduction

Dans le cadre de ce projet Open Data, nous avons développé un site web de simulation permettant d'étudier la propagation d'un virus au sein d'une université virtuelle. L'objectif principal était de créer un environnement réaliste dans lequel des étudiants se déplacent, interagissent et peuvent transmettre un virus selon différentes règles épidémiologiques.

Ce projet combine plusieurs domaines informatiques :

- La modélisation multi-agents, consiste à simuler un système en représentant chaque individu comme un agent autonome dont les interactions produisent des comportements globaux
- la théorie des graphes
- l'analyse de données
- l'intelligence artificielle
- la simulation sanitaire

Le site web permet de visualiser en temps réel l'évolution d'une épidémie au sein d'une université virtuelle. Dans notre application, l'utilisateur peut choisir le nombre d'étudiants à simuler, avec un minimum de 20 étudiants et un maximum de 1000 étudiants générés artificiellement. Grâce à différents paramètres interactifs, l'utilisateur peut modifier le comportement de la simulation et observer les conséquences sur la propagation du virus.

Environnement de travail

On a réalisé le projet en binôme, pendant 8 séances de 4 heures en classe. Nous avons principalement travaillé sur Visual Studio Code, qui nous a permis d'écrire, tester et organiser notre code de manière claire. Pour la gestion du projet, nous avons utilisé Git, afin de sauvegarder régulièrement notre travail, suivre les différentes modifications et faciliter la collaboration entre nous deux. Nous nous sommes également appuyés sur plusieurs outils d'intelligence artificielle, comme ChatGPT, Gemini, Claude et Cursor, tout au long du projet. Ces outils nous ont aidés dans plusieurs aspects : la gestion et l'organisation des données, la compréhension et la correction du code, le développement du site web, ainsi la résolution des erreurs techniques. L'intelligence artificielle a donc été utilisée comme un véritable outil d'accompagnement pendant le projet.

Le projet a été développé avec les outils suivants :

- Python : langage utilisé pour générer les données des étudiants et les convertir en format exploitable par la simulation
- JavaScript : langage principal utilisé pour développer la simulation, gérer les agents, les interactions et la propagation du virus
- HTML / CSS : utilisés pour structurer et styliser l'interface utilisateur

- Canvas API (HTML5) : utilisée pour afficher la simulation en 2D (vue spatiale et graphe des interactions)
- Algorithme A* : utilisé pour le calcul de chemins optimaux et le déplacement intelligent des agents dans l'environnement
- Modèle SEIR : utilisé pour simuler l'évolution de la maladie (Sain, Exposé, Infecté)
- JSON : format utilisé pour stocker les données des étudiants côté JavaScript
- CSV : format utilisé pour importer les données initiales, exporter les résultats de la simulation (Open Data)
- Manipulation du DOM (JavaScript) : utilisée pour créer une interface interactive (sliders, boutons, affichage des statistiques)

Chapitre 1

Description du projet et objectifs

1.1 Objectif principal

L'objectif de ce projet est de simuler la propagation d'un virus au sein d'un établissement universitaire afin de mieux comprendre les mécanismes de transmission entre étudiants. Pour cela, chaque étudiant est modélisé comme un agent autonome possédant des caractéristiques spécifiques :

- un profil individuel (nom, niveau, classe ...)
- un emploi du temps (heures d'arrivée et de départ)
- un niveau de sociabilité influençant ses interactions
- un comportement de déplacement dans l'environnement
- un état de santé (infecté , incubé ou sain)

1.2 Fonctionnement global

La simulation repose sur un système dynamique dans lequel les agents évoluent en temps réel au sein de l'environnement. Le fonctionnement global s'articule autour de plusieurs mécanismes :

- Les agents se déplacent dans l'environnement grâce à un système de navigation basé sur l'algorithme A* (A-Star). Cet algorithme permet de calculer le chemin optimal entre deux points tout en évitant les obstacles. Les agents peuvent ainsi se rendre dans leur salle de classe, circuler dans les couloirs, aller à la cafétéria ou à l'extérieur, quitter l'établissement. Un mécanisme de répulsion des murs est également intégré afin d'éviter les collisions et garantir un déplacement réaliste.
- des interactions sont détectées lorsque deux agents sont proches spatialement
- des contaminations peuvent se produire en fonction : de la distance entre agents, de la probabilité de transmission, du type de zone (classe, couloir, cafétéria, extérieur)

Le modèle épidémiologique utilisé est une version simplifiée du modèle SEIR, permettant de simuler l'évolution de l'état de santé des agents. il repose sur trois états principaux :

- Sain (susceptible) : peut être contaminé,
- Incubé (exposé) : infecté mais non contagieux,

- Infecté : contagieux.
- Rétabli : L'état « rétabli » n'a pas été pris en compte dans cette simulation ; il constitue un axe d'amélioration.

Un agent peut passer de l'état sain à incubé lors d'un contact, puis devenir infecté après une période d'incubation. À l'issue de la simulation, les données générées sont exportées au format CSV. Cela permet d'obtenir des statistiques et de suivre l'évolution de la contamination au cours du temps afin d'analyser la propagation du virus selon différents scénarios.



FIGURE 1.1 –

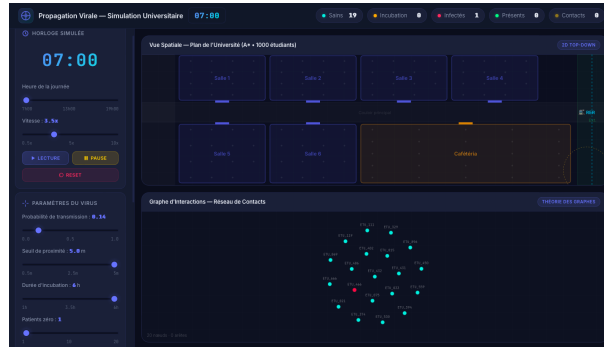


FIGURE 1.2 –

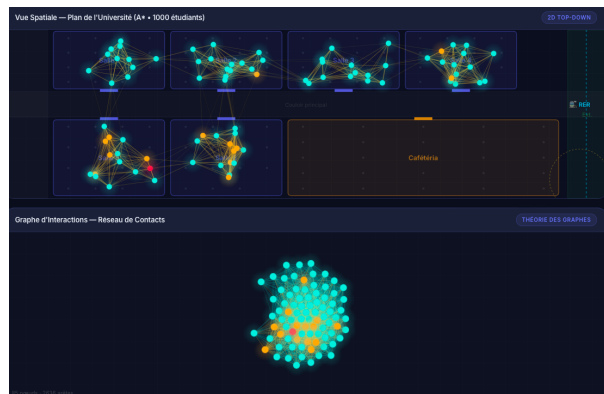


FIGURE 1.3 –

Patients zéro : **1**

11020

Zone × transmission :

- Classe ×0.8
- Couloir ×0.5
- Cafétéria ×1.2
- Extérieur ×0

 POPULATION SIMULÉE

Étudiants actifs : **20**

20135250

Sélectionne aléatoirement parmi les 1000 étudiants du CSV. Modifie la valeur puis relâche pour redémarrer.



TÉLÉCHARGER CSV INSTANTANÉ

 LÉGENDE SEIR

- Sain
- Exposé (incubation)
- Infecté (contagieux)
- Contact $\leq 1m$
- Salle de classe
- Cafétéria
- Cour / Extérieur

FIGURE 1.4 –

Chapitre 2

Bibliothèques, Outils et Technologies

Le projet repose sur plusieurs bibliothèques et technologies permettant de simuler la propagation d'un virus dans une université.

- Python : langage principal utilisé pour développer l'ensemble de la simulation.
- Mesa : framework de simulation multi-agents utilisé pour gérer les étudiants, leurs comportements et leurs interactions.
- NetworkX : bibliothèque utilisée pour modéliser les graphes et les déplacements entre les différentes salles de l'université.
- Pygame : outil utilisé pour l'affichage graphique et la visualisation de la simulation en temps réel.
- Pandas : bibliothèque utilisée pour le traitement, l'analyse et l'exploitation statistique des données générées.
- CSV : format utilisé pour exporter les données de contamination afin de suivre l'évolution du virus et produire des statistiques.
- Algorithme A* : algorithme de recherche de chemin utilisé pour permettre des déplacements intelligents des agents dans l'environnement simulé.
- SQLite : base de données utilisée pour stocker les informations des étudiants, des interactions et des contaminations générées pendant la simulation.
- Intelligence artificielle générative : des outils comme OpenAI ChatGPT, Gemini, Claude et Cursor Cursor ont été utilisés comme aide au développement, à la correction du code, à la gestion des données.

Chapitre 3

Travail réalisé

L’objectif était de créer un environnement dynamique dans lequel plusieurs étudiants interagissent quotidiennement afin d’observer l’évolution de la contamination au fil du temps, de 7h à 18h, sur plusieurs jours voire sur une semaine complète de simulation.

3.1 Création de l’environnement de simulation

La première étape du projet a été la conception de l’université. Un environnement composé de plusieurs salles a été créé afin de représenter les différents lieux fréquentés par les étudiants : salles de cours, couloirs, cafétéria, zones communes. Chaque lieu possède des coordonnées et des connexions permettant aux étudiants de se déplacer dans l’espace simulé. Ces informations sont utilisées par l’algorithme A* afin de calculer automatiquement les chemins les plus courts et les déplacements les plus réalistes entre les différentes zones de l’université.

Afin de rendre la simulation plus réaliste, plusieurs maladies possédant des modes de transmission différents ont été étudiées, notamment la grippe saisonnière, le COVID-19 et l’hantavirus. Chaque pathologie possède ses propres paramètres de contamination, de proximité et d’incubation afin d’observer différentes évolutions possibles de la propagation du virus au sein d’une université simulée.

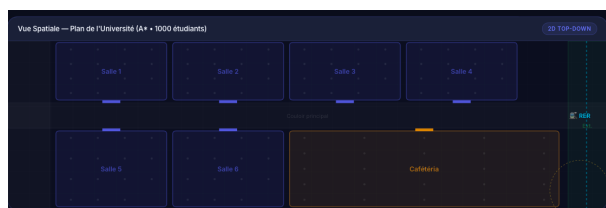


FIGURE 3.1 –

3.2 Génération automatique de la base de données

Un script Python a été développé afin de générer automatiquement plusieurs centaines d’étudiants fictifs. Chaque étudiant possède plusieurs caractéristiques : un identifiant unique, un nom, un prénom, un emploi du temps, un niveau de sociabilité, un niveau

scolaire, un état de santé initial. Les données sont ensuite enregistrées dans une base SQLite. Cette génération automatique permet : d’éviter une saisie manuelle trop longue, d’obtenir des profils variés, de rendre la simulation plus réaliste, de produire davantage de données statistiques. La base de données contient également : les déplacements, les interactions entre étudiants, les contaminations détectées, l’évolution de l’état sanitaire des agents.

Plusieurs fichiers du projet sont dédiés à cette génération et à l’initialisation de la base de données, notamment : `init_dynamic_db.py`, `dynamic_simulation.db`, `simulation_virus.db`.

ID	Nom_Prenom	Niveau	Classe	Adresse	Profil	Exposant	Moyenne_Accueil	Moyenne_Depart	Indice_Socialite	Statut_Sante	Statut_Labor	Temps_Isolation	Restant	Contacts_Total	Moins_Export
ETU_760	Chloe Girard	L3	Salv-3	Externeur	08-10	1631	0.99	0	2	Infecte	0	0	0	0	0
ETU_466	Camille Petit	L1	Salv-3	Couloir	08-38	17-12	0.10	0	2	Infecte	0	0	0	0	0
ETU_539	Nathan Vincent	Master	Salv-6	Couloir	07-49	16-33	0.67	0	2	Infecte	0	0	0	0	0
ETU_265	Julie Leroy	L2	Salv-4	Externeur	08-37	17-45	0.24	0	2	Infecte	0	0	0	0	0
ETU_110	Paul Fournier	L3	Salv-4	Couloir	07-48	16-04	0.17	0	0	Sain	0	0	0	0	0
ETU_790	Antoine Martin	L3	Salv-2	Couloir	08-43	17-59	0.89	0	0	Sain	0	0	0	0	0
ETU_370	Louis Bertrand	Master	Salv-6	Externeur	08-21	16-38	0.04	0	0	Sain	0	0	0	0	0
ETU_322	Antoine Bertrand	L3	Salv-1	Cafeteria	07-37	16-56	0.86	0	0	Sain	0	0	0	0	0
ETU_332	Ennna Lefebvre	L3	Salv-4	Couloir	08-44	16-20	0.27	0	0	Sain	0	0	0	0	0
ETU_362	Arthur LeFebvre	L1	Salv-4	Cafeteria	08-16	17-54	0.60	0	0	Sain	0	0	0	0	0
ETU_267	Ennna Roux	Master	Salv-2	Cafeteria	08-56	16-13	0.83	0	0	Sain	0	0	0	0	0
ETU_266	Clara Simon	L1	Salv-5	Cafeteria	08-38	16-03	0.47	0	0	Sain	0	0	0	0	0
ETU_117	Marion Dubois	L1	Salv-5	Cafeteria	08-31	16-41	0.48	0	0	Sain	0	0	0	0	0
ETU_769	Thomas Lefebvre	L1	Salv-1	Externeur	08-53	16-12	0.59	0	0	Sain	0	0	0	0	0
ETU_363	Paul Richard	L2	Salv-5	Externeur	08-13	16-41	0.39	0	0	Sain	0	0	0	0	0
ETU_038	Julien Fournier	L1	Salv-2	Cafeteria	07-58	16-29	0.97	0	0	Sain	0	0	0	0	0
ETU_868	Samuel Simon	L1	Salv-4	Couloir	07-51	16-08	0.84	0	0	Sain	0	0	0	0	0
ETU_1005	Paul Vincent	Master	Salv-6	Couloir	07-51	17-47	0.93	0	0	Sain	0	0	0	0	0
ETU_447	Paul Fournier	Master	Salv-2	Externeur	08-59	16-56	0.74	0	0	Sain	0	0	0	0	0
ETU_368	Maria Leroy	L2	Salv-4	Couloir	08-01	16-45	0.16	0	0	Sain	0	0	0	0	0
ETU_872	Lila Thomas	Master	Salv-2	Couloir	08-58	17-17	0.57	0	0	Sain	0	0	0	0	0
ETU_470	Julien Lambert	L2	Salv-3	Couloir	08-39	16-09	0.39	0	0	Sain	0	0	0	0	0
ETU_789	Louisa Morel	L2	Salv-4	Externeur	08-00	16-37	0.33	0	0	Sain	0	0	0	0	0
ETU_364	Lila Roux	L3	Salv-2	Cafeteria	08-34	17-26	0.40	0	0	Sain	0	0	0	0	0
ETU_366	Mathilde Muller	L1	Salv-5	Cafeteria	08-37	16-24	0.88	0	0	Sain	0	0	0	0	0
ETU_342	Arthur Fontaine	L1	Salv-6	Externeur	08-31	17-30	0.54	0	0	Sain	0	0	0	0	0
ETU_538	Reyan Richard	L1	Salv-5	Cafeteria	08-10	16-30	0.54	0	0	Sain	0	0	0	0	0
ETU_817	Ennna Roux	L2	Salv-2	Cafeteria	07-58	16-29	0.38	0	0	Sain	0	0	0	0	0
ETU_338	Gabriel Simon	Master	Salv-6	Externeur	08-47	16-10	0.89	0	0	Sain	0	0	0	0	0
ETU_086	Inna Fontaine	L3	Salv-2	Externeur	08-24	17-41	0.48	0	0	Sain	0	0	0	0	0
ETU_418	Nathan Roux	Master	Salv-4	Couloir	08-00	16-06	0.36	0	0	Sain	0	0	0	0	0
ETU_360	Samuel Morel	Master	Salv-4	Externeur	08-52	16-12	0.48	0	0	Sain	0	0	0	0	0
ETU_317	Paul Girard	L2	Salv-2	Couloir	08-21	16-13	0.29	0	0	Sain	0	0	0	0	0
ETU_876	Ennna Leroy	Master	Salv-6	Externeur	08-10	16-35	0.26	0	0	Sain	0	0	0	0	0
ETU_485	Louisa Simon	L3	Salv-4	Cafeteria	08-18	17-15	0.51	0	0	Sain	0	0	0	0	0
ETU_858	Maria Dupont	L1	Salv-6	Cafeteria	08-41	16-32	0.49	0	0	Sain	0	0	0	0	0
ETU_814	Ennna Bertrand	L1	Salv-2	Externeur	08-52	17-59	0.18	0	0	Sain	0	0	0	0	0
ETU_014	Camille Laurent	Master	Salv-3	Couloir	08-37	17-31	0.41	0	0	Sain	0	0	0	0	0

FIGURE 3.2 –

3.3 Gestion des agents avec Mesa

Le framework Mesa a été utilisé afin de créer les agents de la simulation. Dans le projet, chaque étudiant est considéré comme un agent autonome capable : de se déplacer, d’interagir avec d’autres agents, de changer d’état de santé, de suivre un emploi du temps. Chaque agent possède son propre comportement, ce qui signifie que les étudiants n’ont pas tous les mêmes déplacements, les mêmes interactions sociales ni la même fréquence de contact avec les autres agents. Par exemple : certains étudiants se déplacent davantage, certains interagissent plus souvent, certains restent plus longtemps dans certaines zones.

3.4 Déplacements intelligents et modélisation avec NetworkX

Les déplacements des étudiants ont été modélisés sous forme de graphe grâce à la bibliothèque NetworkX. Chaque salle représente un nœud du graphe et les connexions entre les salles représentent les chemins possibles. Cela permet : d’éviter des déplacements incohérents, d’améliorer le réalisme de la simulation, de fluidifier les mouvements des agents. Les étudiants peuvent ainsi se déplacer automatiquement entre : les salles de cours, les couloirs, la cafétéria, les espaces communs.

3.5 Implémentation du modèle SEIR

La propagation du virus repose sur le modèle SEIR. Cependant, l'état rétabli n'a pas été entièrement implémenté dans cette version du projet, car les étudiants restent infectés pendant toute la durée de la simulation.

- S : Sain
- E : Exposé
- I : Infecté
- R : Rétabli.

Lorsqu'un étudiant infecté se trouve proche d'un étudiant sain, une probabilité de transmission est calculée. Après contamination : une période d'incubation est simulée, l'étudiant devient infecté.

3.6 Détection des interactions et contaminations

Pendant la simulation, les interactions entre étudiants sont analysées en continu. Le système vérifie : la distance entre les agents, le temps passé ensemble, la zone dans laquelle ils se trouvent. Lorsqu'une interaction à risque est détectée, une contamination peut être enregistrée selon une probabilité définie. Toutes ces informations sont stockées dans la base de données afin de conserver un historique complet de la simulation.

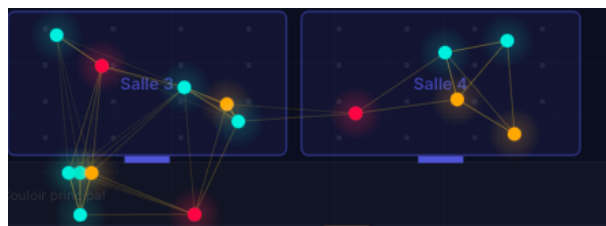


FIGURE 3.3 –

3.7 Visualisation graphique avec Pygame

Une interface graphique a été développée avec Pygame afin de visualiser la simulation en temps réel. L'affichage permet : de suivre les déplacements des étudiants, de visualiser les zones fréquentées, d'observer les contaminations, de distinguer les états de santé grâce à différentes couleurs. Par exemple : bleu : sain / orange : exposé / rouge : infecté.

3.8 Export des données au format CSV

Lorsque l'utilisateur le souhaite, les données de la simulation peuvent être exportées automatiquement au format CSV. Les fichiers générés contiennent les données précédentes, auxquelles s'ajoutent la mise à jour de l'état de santé des étudiants, le nombre de contaminations ainsi que le nombre total d'étudiants fixé dans la simulation. Parmi les fichiers utilisés dans le projet, on retrouve notamment : `visualize_infections.py`, `animate_simulation.py`, les fichiers CSV générés automatiquement pendant les simulations.

Chapitre 4

Détails Techniques (Analyse des fichiers)

4.1 Fichier graph.js

Le fichier graph.js gère la visualisation des interactions sociales sous forme de graphe dynamique. Il permet d'observer comment les contacts entre étudiants évoluent au cours de la simulation.

- **Objectif** : Visualiser les agents sous forme de nœuds, représenter les interactions sous forme d'arêtes, suivre l'évolution des contaminations, analyser la structure des contacts.
- **Synchronisation** : La méthode `syncWithSimulation()` maintient le graphe à jour.
- **Algorithme de forces** : Répulsion, attraction, recentrage et amortissement.
- **Rendu** : Affichage d'un fond sombre, grille, arêtes colorées selon l'état sanitaire, halo sur les nœuds actifs.

4.2 Fichier map.js

Définit l'environnement spatial (structure de l'école).

- **Structure** : Repère 1000x600, échelle 1m = 20 unités. Salles, cafétéria, couloirs, zone extérieure.
- **Gestion des salles** : Position, taille, type, porte (`getDoor()`).
- **isWalkable()** : Détermine si un agent peut se déplacer à une position donnée.
- **wallRepulsion()** : Empêche de traverser les murs.

4.3 Fichier pathfinding.js

Navigation intelligente A*.

- **Discrétisation** : Grille de cellules de 10 unités (0,5m).
- **find()** : Calcule le coût du trajet (distance parcourue + estimation restante).
- **Optimisation** : Simplification du chemin et vérification de la visibilité directe.

4.4 Fichier `renderer.js`

Visualisation spatiale 2D.

- **resize()** : Adaptation responsive.
- **render()** : Dessine fond, zones, couloirs, salles, portes, sortie RER, interactions et agents.
- **Agents** : Bleu (sain), orange (incubé), rouge (infecté).
- **Référence** : Cercle de distance de contamination.

4.5 Fichier `simulation.js`

Cœur de la simulation.

- **init()** : Prépare la population et les patients zéro.
- **tick()** : Progression du temps simulé.
- **_processContamination()** : Transmission selon zone (risque élevé en cafétéria, nul à l'extérieur).
- **exportCSV()** : Génération des données Open Data.

4.6 Fichier `ui.js`

Interface utilisateur.

- **Contrôles** : Sliders (vitesse, transmission, distance, incubation, population), boutons (play, pause, reset, export).
- **update()** : Affichage de l'heure et des statistiques en temps réel.

4.7 Fichier `main.js`

Point d'entrée. Orchestre les modules (Simulation, SpatialRenderer, GraphRenderer, UIController) et gère la boucle d'animation (requestAnimationFrame).

4.8 Scripts Python (Génération et Conversion)

- **students.csv** : Génération de noms, niveaux, horaires, sociabilité (0-1), état initial.
- **students_data.js** : Conversion CSV vers JSON pour JavaScript.

4.9 Fichier `index.html`

Structure HTML5 de la page : En-tête (stats), Panneau de contrôle (aside), Zone de visualisation (main).

Chapitre 5

Bilan

5.1 Difficultés rencontrées

Problème 1 : Modélisation de la contamination

L'une des premières difficultés du projet a été de représenter une propagation du virus suffisamment réaliste. Au début, les contaminations étaient trop rapides ou incohérentes par rapport à une véritable épidémie. **Solution** : Le modèle SEIR a été mis en place afin d'introduire plusieurs états de santé : sain, exposé, infecté. L'état « rétabli » n'a donc pas été utilisé dans la version finale du projet.

Problème 2 : Déplacement des agents

Les premiers déplacements des étudiants posaient plusieurs problèmes : certains agents traversaient les murs, certains déplacements étaient incohérents, plusieurs agents restaient bloqués. **Solution** : L'algorithme A* a été implémenté afin de calculer automatiquement les chemins possibles entre les différentes salles de l'université. Les déplacements sont ainsi devenus plus fluides et réalistes.

Problème 3 : Génération et qualité des données

La création de données réalistes pour les étudiants a également été compliquée. Certaines données générées automatiquement étaient incohérentes. **Solution** : Des scripts spécifiques du projet, comme `init_dynamic_db.py`, ont été modifiés plusieurs fois afin de corriger la génération des étudiants et des données de simulation. Plusieurs problèmes ont été rencontrés dans les tables SQLite, notamment des incohérences entre les emplois du temps, les salles et les déplacements des agents. Les fichiers d'export CSV ainsi que les scripts `dynamic_engine.py` et `visualize_infections.py` ont également été ajustés afin de récupérer correctement les données de contamination, les états des étudiants et les statistiques générées pendant la simulation.

Problème 4 : Utilisation des intelligences artificielles

Une grande difficulté a été d'apprendre à utiliser efficacement les intelligences artificielles comme ChatGPT, Gemini, Claude ou Cursor. Au début : les prompts étaient parfois trop vagues, certaines IA donnaient des solutions incorrectes. Il fallait également réussir à choisir la meilleure IA selon les besoins : génération de code, correction d'erreurs, aide à la rédaction, génération de données, optimisation du projet. **Solution** : De nombreux essais.

5.2 Perspectives et améliorations possibles

- **Amélioration des comportements des agents** : Étudiants isolés, groupes d'amis, temps variable.
- **Paramétrage complet du plan** : Ajouter/supprimer des salles, modifier couloirs, créer plusieurs bâtiments.
- **Paramétrage des salles** : Capacité, ventilation, densité, risque.
- **Emploi du temps individuel** : Planning différent pour chaque étudiant pour plus de réalisme.
- **Ajout de mesures sanitaires** : Masque, distanciation, fermeture de salles, quarantaine, vaccination.
- **Amélioration de l'interface graphique** : Carte détaillée, graphiques dynamiques, menus interactifs.

5.3 Simulations supplémentaires possibles

Le projet pourrait être adapté à : un hôpital, un centre commercial, les transports en commun, une pandémie à grande échelle ou une prison.

Bibliographie

Bibliothèques et documentations techniques

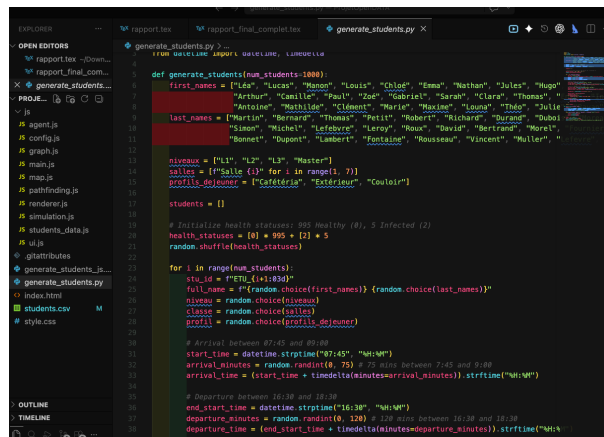
- **Python Documentation** : Langage principal.
<https://www.python.org/doc/>
- **Mesa Documentation** : Simulation multi-agents.
<https://mesa.readthedocs.io/>
- **NetworkX Documentation** : Modélisation des graphes.
<https://networkx.org/documentation/stable/>
- **Pygame Documentation** : Interface graphique.
<https://www.pygame.org/docs/>
- **Pandas Documentation** : Traitement des données CSV.
<https://pandas.pydata.org/docs/>
- **SQLite Documentation** : Gestion de la base de données.
<https://www.sqlite.org/docs.html>

Outils d'intelligence artificielle utilisés

- **ChatGPT - OpenAI** : Utilisé pour l'aide à la programmation, à la rédaction du rapport et à la résolution de problèmes techniques.
<https://chat.openai.com/>
- **Gemini** : Utilisé pour certaines recherches et comparaisons techniques.
<https://gemini.google.com/>
- **Claude** : Utilisé comme assistance complémentaire pour le développement et la reformulation.
<https://claude.ai/>
- **Cursor** : Utilisé pour l'aide au développement du code Python et au débogage du projet.
<https://cursor.com/>

Annexes

Extrait du code Python : Figure 5.1



```
1 from os.path import join, dirname, realpath
2
3 def generate_students(num_students=1000):
4     first_names = ["Eva", "Lucas", "Marianne", "Louis", "Emile", "Mathieu", "Julie", "Hugo",
5                   "Arthur", "Camille", "Paul", "Zoe", "Gabriel", "Sarah", "Clara", "Thomas",
6                   "Antoine", "Mathilde", "Clément", "Marie", "Maxime", "Léon", "Théo", "Julie",
7                   "Martin", "Bernard", "Thomas", "Pascal", "Robert", "Richard", "Dorian", "Romain",
8                   "Simon", "Michele", "Léonore", "Léon", "Romain", "David", "Marion", "Maurice",
9                   "Bonnet", "Dupont", "Lambert", "Fontaine", "Rousseau", "Vincent", "Müller", "
10
11     last_names = ["Léon", "Léon", "Léon", "Léon"]
12     salles = [{"salle": i} for i in range(1, 7)]
13     profiles_dejeune = ["Interdiction", "Extérieur", "Coulée"]
14     students = []
15
16     # Initialize health statuses: 995 Healthy (0), 5 Infected (1)
17     health_statuses = [0] * 995 + [1] * 5
18     random.shuffle(health_statuses)
19
20     for i in range(num_students):
21         stu_id = f"STU-{i+1:04d}"
22         full_name = f"{random.choice(first_names)} {random.choice(last_names)}"
23         niveau = random.choice(niveaux)
24         classe = random.choice(salles)
25         profil = random.choice(profiles_dejeune)
26
27         # Arrival between 07:30 and 08:00
28         start_time = datetime.strptime("07:45", "%H:%M")
29         arrival_minutes = random.randint(0, 75) # 75 mins between 7:45 and 8:00
30         arrival_time = (start_time + timedelta(minutes=arrival_minutes)).strftime("%H:%M")
31
32         # Departure between 16:30 and 18:30
33         end_start_time = datetime.strptime("16:30", "%H:%M")
34         departure_minutes = random.randint(0, 120) # 120 mins between 16:30 and 18:30
35         departure_time = (end_start_time + timedelta(minutes=departure_minutes)).strftime("%H:%M")
```

FIGURE 5.1 –

Capture de la base SQLite : Figure 3.2

Capture de la simulation graphique : Figure 1.2

Paramètres de simulation : Figure 1.4

Conclusion

Ce projet de simulation épidémiologique démontre qu'il est possible de reproduire des comportements humains complexes grâce à la combinaison de Python, des graphes, de la modélisation multi-agents et de l'intelligence artificielle. Grâce à une base de données de 1000 étudiants et à une navigation réaliste, la simulation permet d'observer la formation de chaînes de contamination et d'analyser les interactions sociales dans un environnement fermé. Le projet met également en avant une nouvelle manière de développer un site web complexe grâce au Prompt Engineering, où l'humain se concentre principalement sur la logique et l'analyse tandis que l'IA assiste la programmation technique. Enfin, cette approche montre que les outils numériques peuvent devenir de véritables laboratoires virtuels capables d'aider à comprendre et anticiper des crises sanitaires réelles.

5.4 Conclusion

Ce projet a permis de développer une simulation de propagation virale dans une université virtuelle en combinant plusieurs domaines de l'informatique et de la modélisation. La simulation repose notamment sur :

- la modélisation multi-agents avec Mesa ;
- la théorie des graphes avec NetworkX ;
- des algorithmes de déplacements intelligents comme A* ;
- l'exploitation de bases de données SQLite ;
- l'analyse statistique grâce aux fichiers CSV ;
- l'utilisation d'outils d'intelligence artificielle pour accompagner le développement du projet.

Le projet a également permis de mieux comprendre : les mécanismes de propagation d'une épidémie ; l'importance des interactions sociales ; l'impact des déplacements dans un environnement fermé ; la difficulté de créer une simulation réaliste et exploitable. Cette expérience a permis de développer des compétences en : programmation Python ; gestion de données ; simulation multi-agents ; visualisation graphique ; travail de recherche et d'analyse.